**Bahasa Pemrograman Tingkat Rendah (MWCA010)**

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

# Arsitektur Mikroprosesor 8086

Pertemuan 2

---

Dosen Pengampu

**Cecep Suwanda, S.Si., M.Kom.**

Semester Ganjil

# Capaian dan Tujuan Pembelajaran

## Sub-CPMK 1.1

Menjelaskan arsitektur dasar mikroprosesor 8086 (*Execution Unit* dan *Bus Interface Unit*), peran bus alamat/data/kontrol, serta tahapan siklus instruksi (*fetch–decode–execute*).

**CPMK yang didukung: CPMK01** ▪ **CPL yang didukung: CPL02**

## Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan kedudukan 8086 sebagai dasar keluarga x86 beserta evolusinya;
2. membedakan arsitektur internal dan arsitektur eksternal mikroprosesor;
3. menguraikan pembagian tugas BIU dan EU serta peran *instruction queue*;
4. menyebutkan fungsi bus alamat, data, dan kontrol beserta sinyal utamanya;
5. menelusuri satu instruksi melalui *fetch–decode–execute–store*.

# Peta Materi

1. **Evolusi Mikroprosesor dan Kedudukan 8086**
   dari 4004 sampai 8086; mengapa 8086 masih dipelajari; arsitektur internal dan eksternal
2. **Bus Interface Unit dan Execution Unit**
   diagram blok 8086; pembagian tugas unit; *instruction queue* dan *pipelining*; alamat fisik
3. **Bus Eksternal dan Siklus Instruksi**
   bus alamat, data, kontrol; *multiplexing* AD0–AD15; siklus *fetch–store*

## Praktik pada pertemuan ini

Menggambar diagram blok BIU/EU dari ingatan, lalu menelusuri siklus instruksi dengan *Single Step* pada Emu8086.

BAGIAN

# Evolusi Mikroprosesor dan Kedudukan 8086

# Dari 4004 ke 8086

| Tahun | Prosesor | Bus data | Bus alamat | Catatan |
|---|---|---|---|---|
| 1971 | Intel 4004 | 4 bit | 12 bit | Mikroprosesor komersial pertama, untuk kalkulator |
| 1974 | Intel 8080 | 8 bit | 16 bit | Ruang alamat 64 KB; *microcomputer* awal |
| 1978 | Intel 8086 | 16 bit | 20 bit | Ruang alamat 1 MB; dasar keluarga x86 |
| 1979 | Intel 8088 | 8 bit (luar) | 20 bit | Inti 16-bit, bus luar 8-bit; dipakai IBM PC |
| 1982 | Intel 80286 | 16 bit | 24 bit | Menambah *protected mode*, 16 MB alamat |

Setiap generasi menambah lebar kata, lebar bus alamat, dan jumlah transistor. Varian 8088 berinti 16 bit hanya menyempitkan bus luarnya menjadi 8 bit.

Sumber: Bab IV, Bagian 1.1 — Tabel 4.1

# Mengapa 8086 Masih Dipelajari

1. **Sederhana namun lengkap.** Sudah memiliki seluruh konsep penting prosesor modern: register, bus, *flag*, segmentasi, dan *pipeline* sederhana, tanpa mekanisme rumit seperti *cache* berlapis.

2. **Mudah diamati.** Emu8086 memperlihatkan isi register, memori, dan *flag* pada setiap langkah eksekusi, sehingga konsep abstrak dapat diverifikasi sendiri.

3. **Akar keluarga x86.** Prosesor x86 modern tetap kompatibel ke belakang; program 8086 mode real masih dapat dijalankan melalui mode kompatibilitas.

## Intinya

Usianya lebih dari empat dekade, tetapi 8086 adalah model paling jernih untuk memahami cara kerja sebuah prosesor.

# Arsitektur Internal dan Eksternal

## Arsitektur internal (*internal architecture*)

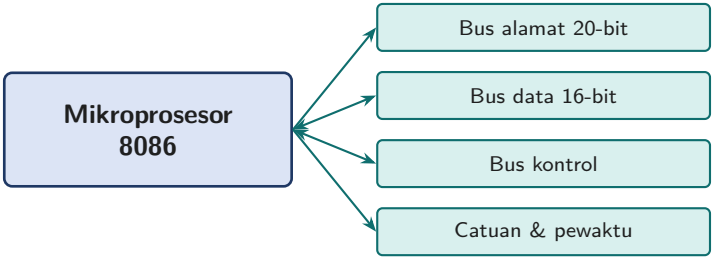
Susunan unit fungsional **di dalam chip**: register, ALU, unit pengendali, dan saluran data internal. Menjelaskan *bagaimana* prosesor mengolah sebuah instruksi.

## Arsitektur eksternal (*external architecture*)

Tampak **luar chip**: jumlah dan fungsi pin, kelompok bus, serta sinyal kendali ke memori dan perangkat I/O. Menjelaskan *bagaimana* prosesor berbicara dengan dunia luar.

Programmer assembly lebih sering berhadapan dengan arsitektur internal (menuliskan nama register); perancang sistem digital lebih banyak bekerja pada arsitektur eksternal (pin `A0`–`A19`, `AD0`–`AD15`, sinyal kendali).

# Cara Pandang Sederhana: Kotak Hitam




Ke luar, prosesor hanya berhubungan melalui empat kelompok saluran tersebut. Setiap instruksi yang dijalankan pasti melibatkan pertukaran informasi lewat salah satunya.

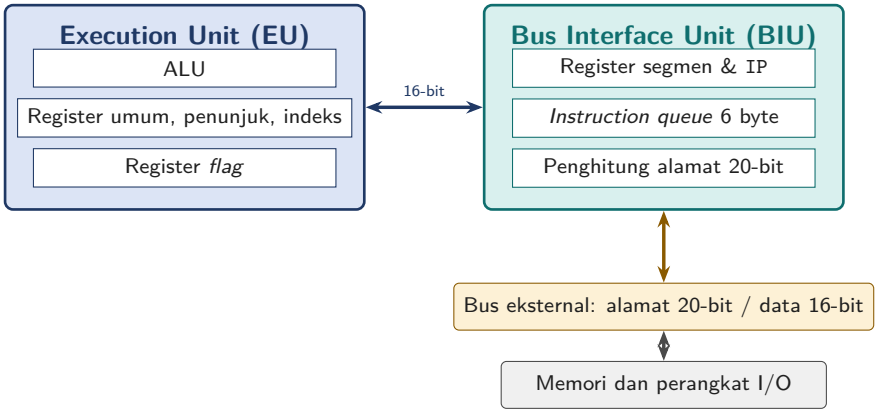
# Ringkasan Bagian 1

- Mikroprosesor berkembang dari Intel 4004 (4 bit) sampai 8086 (16 bit; bus alamat 20 bit) yang menjadi dasar keluarga x86.
- Varian 8088 berinti 16 bit dengan bus luar 8 bit dipakai pada IBM PC.
- 8086 dipelajari karena sederhana namun lengkap, mudah diamati pada emulator, dan menjadi akar keluarga x86.
- Arsitektur dipandang dari dua sisi: **internal** (register, ALU, unit di dalam chip) dan **eksternal** (pin, bus, sinyal kendali).

BAGIAN

# Bus Interface Unit dan Execution Unit

# Diagram Blok Internal 8086




EU di kiri, BIU di kanan, dihubungkan bus internal 16 bit. Hanya BIU yang menyentuh pin luar chip.

# Tugas Masing-masing Unit

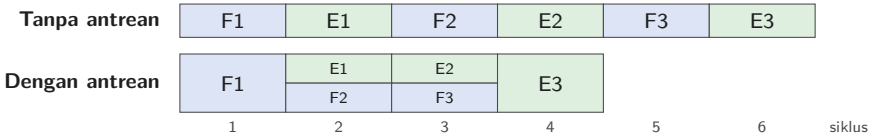
| Aspek | Bus Interface Unit (BIU) | Execution Unit (EU) |
|---|---|---|
| Tugas utama | Mengambil instruksi dan data; menangani seluruh lalu lintas bus | Menerjemahkan dan menjalankan instruksi |
| Register | `CS`, `DS`, `SS`, `ES`, `IP`, *instruction queue* | `AX`–`DX`, `SP`, `BP`, `SI`, `DI`, *flag* |
| Akses bus luar | Wewenang tunggal | Tidak punya; harus meminta melalui BIU |
| Perhitungan alamat | Alamat fisik 20-bit dari pasangan segmen:offset | Hanya *offset* (*effective address*) |

## Yang perlu diingat

Ketika sebuah instruksi memerlukan operand dari memori, EU mengirim permintaan alamat ke BIU, lalu BIU menjalankan siklus bus dan menyerahkan datanya kembali.

# Instruction Queue dan Pipelining

Selama EU masih sibuk, BIU memakai waktu luang bus untuk mengisi *instruction queue* enam byte.




F = *fetch* (BIU) ▪ E = *execute* (EU) ▪ Tanpa antrean: 6 siklus; dengan antrean: 4 siklus.

# Peringatan: Queue Flush

## *Pipeline* pada 8086 hanya satu tingkat

Antreannya hanya enam byte. Ketika EU menjalankan instruksi lompatan (`JMP`, `CALL`), isi antrean sudah tidak relevan sehingga harus dikosongkan. Peristiwa ini disebut **queue flush**.

- *Queue flush* menghapus keuntungan *pipeline* untuk instruksi tersebut.
- Akibatnya, program yang penuh lompatan berjalan lebih lambat daripada program lurus dengan panjang yang sama.

Sumber: Bab IV, Bagian 2.3

# Peran Segmentasi pada BIU

BIU mengubah pasangan **segmen:offset** menjadi alamat fisik 20-bit. Register segmen hanya 16 bit, sedangkan bus alamat selebar 20 bit.

$$\text{Alamat fisik} = (\text{register segmen} \times 16) + \textit{offset}$$

## Contoh

`CS` = `1234h`, `IP` = `0010h`

`12340h` + `0010h` = **`12350h`**

Mengalikan dengan 16 sama dengan menggeser nilai segmen satu digit heksadesimal ke kiri.

Rumus inilah alasan 8086 mampu menjangkau 1 MB memori meskipun register-nya hanya 16 bit.

# Sekilas Register Internal

| Kelompok | Anggota | Kegunaan utama |
|---|---|---|
| Register umum | `AX`, `BX`, `CX`, `DX` | Menyimpan data dan hasil operasi aritmetika |
| Register penunjuk | `SP`, `BP` | Puncak *stack* dan kerangka data prosedur |
| Register indeks | `SI`, `DI` | Sumber dan tujuan pada operasi string |
| Register segmen | `CS`, `DS`, `SS`, `ES` | Nomor segmen kode, data, *stack*, dan tambahan |
| Penunjuk instruksi | `IP` | *Offset* instruksi berikutnya |
| Register *flag* | `CF`, `ZF`, `SF`, ... | Kondisi hasil operasi terakhir |

Register segmen, `IP`, dan *instruction queue* berada di BIU; sisanya berada di EU. Rincian setiap register dibahas pada Bab V.

# Ringkasan Bagian 2

- 8086 terbagi menjadi BIU dan EU yang bekerja secara paralel.
- BIU: register segmen, `IP`, penghitung alamat 20-bit, dan *instruction queue* enam byte; pemegang tunggal bus luar.
- EU: ALU, register umum/penunjuk/indeks, dan register *flag*; sama sekali tidak terhubung ke pin luar.
- Antrean memungkinkan *pipelining* (6 siklus menjadi 4 siklus); alamat fisik = (segmen $\times$ 16) + *offset*.

BAGIAN

# Bus Eksternal dan Siklus Instruksi

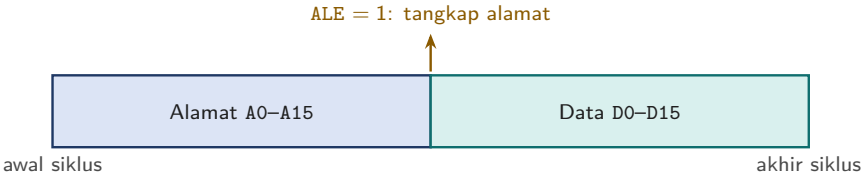
# Tiga Kelompok Bus

Bus adalah sekumpulan penghantar yang dipakai bersama oleh prosesor, memori, dan perangkat I/O.

- **Bus alamat** (*address bus*) — 20 bit, yaitu `A0`–`A19`; membawa alamat lokasi yang akan diakses. Menjangkau $2^{20}$ byte = 1 MB dan bersifat **satu arah** (prosesor → memori/I/O).
- **Bus data** (*data bus*) — 16 bit, yaitu `AD0`–`AD15`; membawa data yang sedang dibaca atau ditulis, dan bersifat **dua arah**.
- **Bus kontrol** (*control bus*) — mengatur *jenis* dan *arah* pertukaran data: baca atau tulis, memori atau perangkat I/O, dan kapan perangkat siap melayani.

# *Multiplexing* AD0–AD15

Kemasan 8086 hanya **40 pin**, sehingga sebagian pin dipakai bergantian: AD0–AD15 membawa alamat pada awal siklus bus, lalu membawa data.



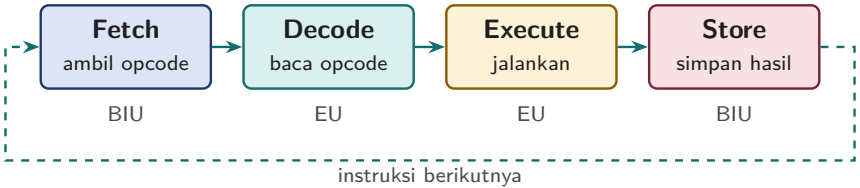

Karena alamat hanya muncul sebentar, rangkaian luar bernama *latch* harus menangkapnya. Sinyal ALE (*Address Latch Enable*) memberi tahu kapan saat yang tepat.

# Sinyal Kendali Utama 8086

| Sinyal | Arah | Fungsi |
|---|---|---|
| ALE | Keluaran | Menandai bahwa AD0–AD15 sedang membawa alamat |
| M/IO | Keluaran | Sasaran akses: tinggi untuk memori, rendah untuk perangkat I/O |
| RD | Keluaran | Aktif rendah; prosesor sedang melakukan operasi baca |
| WR | Keluaran | Aktif rendah; prosesor sedang melakukan operasi tulis |
| READY | Masukan | Perangkat lambat sudah selesai; bila rendah, prosesor menambah *wait state* |
| RESET | Masukan | Kembali ke keadaan awal; CS diisi FFFFh dan IP diisi 0000h |

Sinyal MN/MX memilih mode minimum atau maksimum. Seluruh contoh pada buku ini dapat dipandang berjalan pada mode minimum.

# Siklus Instruksi 8086




**Fetch**: BIU menghitung alamat dari `CS:IP` dan mengisi antrean ▪ **Decode**: EU mengenali opcode ▪ **Store**: hanya memerlukan siklus bus bila tujuannya memori.

# Menelusuri Satu Instruksi

Instruksi `MOV AX, 1234h` berukuran tiga byte dengan opcode `B8h` diikuti `34h` dan `12h` secara *little endian*. Andaikan `CS` = `1000h`, `IP` = `0000h`.

| No. | Tahap | Alamat | IP | Kejadian |
|---|---|---|---|---|
| 1 | Fetch | `10000h` | `0003h` | BIU membaca `B8h`, `34h`, `12h`; antrean: `B8 34 12h` |
| 2 | Decode | — | `0003h` | EU mengenali opcode: `MOV reg16, imm16` |
| 3 | Execute | — | `0003h` | EU menyusun `1234h`, lalu menulis ke `AX` |
| 4 | Store | — | `0003h` | Tujuan register: tidak ada siklus bus |

`IP` bertambah tiga — panjang instruksi menentukan kenaikannya. Bandingkan `MOV hasil1, AX` yang tujuannya di memori: BIU harus menjalankan siklus bus tulis.

Sumber: Bab IV, Bagian 3.3 — Tabel 4.6

# Ringkasan Bagian 3

- Tiga kelompok bus: alamat 20-bit (satu arah), data 16-bit (dua arah), dan kontrol.
- Pin `AD0`–`AD15` dimultipleks; alamat ditangkap *latch* dengan bantuan sinyal `ALE`.
- Siklus instruksi: *fetch* (BIU) $\rightarrow$ *decode* (EU) $\rightarrow$ *execute* (EU) $\rightarrow$ *store* (BIU).
- Tahap *store* hanya memerlukan siklus bus bila tujuannya berada di memori.

# Contoh 1: Menelusuri Siklus Instruksi

Program sesingkat mungkin agar setiap langkah dapat diamati tanpa kehilangan jejak.

```
1  .MODEL SMALL
2  .STACK 100h
3  .DATA
4      nilai_awal DW 1234h
5  .CODE
6  START:
7      MOV AX, @DATA
8      MOV DS, AX
9      MOV AX, 1234h     ; B8 34 12
10     MOV AH, 4Ch
11     MOV AL, 00h
12     INT 21h
13 END START
```

## Panjang instruksi

```
1000:0000  MOV AX, @DATA  3
1000:0003  MOV DS, AX     2
1000:0005  MOV AX, 1234h  3
1000:0008  MOV AH, 4Ch    2
1000:000A  MOV AL, 00h    2
1000:000C  INT 21h        2
```

`IP` bertambah menurut panjang instruksi: 3 byte $\rightarrow +3$, 2 byte $\rightarrow +2$.

Dua instruksi 2 byte dapat berada dalam antrean enam byte sekaligus.

Instruksi yang membaca variabel memicu siklus bus tambahan.

`code/asm/siklus-dasar.asm`

# Contoh 2: Dua Cara Menjumlahkan

```
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    ; cara 1: operand di register
    MOV AX, 1234h
    MOV BX, 5678h
    ADD AX, BX
    MOV hasil1, AX
    ; cara 2: operand di memori
    MOV AX, bil1
    ADD AX, bil2
    MOV hasil2, AX
    MOV AH, 4Ch
    INT 21h
```

`.DATA`: `bil1 1234h, bil2 5678h, hasil1 0h, hasil2 0h`.

## Keluaran

```
hasil1 = 68ACh
hasil2 = 68ACh
```

Cara 1 (register): **11 siklus**.

Cara 2 (memori): **29 siklus**.

Penjumlahannya sama; yang bertambah adalah dua siklus bus pada tahap *store*.

`code/asm/jumlah-dua-cara.asm`

# Praktikum: Diagram Blok dan Penelusuran

## Judul praktikum

Menggambar diagram blok 8086 dan menelusuri siklus instruksi dengan *Single Step* ▪ **Sub-CPMK:** 1.1 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

### Alat dan bahan

- Komputer dengan Emu8086 terpasang
- `siklus-dasar.asm` dan `jumlah-dua-cara.asm`
- Kertas kosong dan alat tulis
- Lembar jurnal praktikum

### Langkah kerja

1. Gambar diagram blok 8086 dari ingatan, lalu koreksi
2. Catat `CS`, `IP`, `AX`, `DS` mula-mula
3. Tekan *Single Step*, catat perubahan tiap langkah
4. Bandingkan panjang instruksi dengan hasil *disassembly*

**Dikumpulkan:** gambar diagram blok beserta koreksinya, tabel pengamatan `CS`/`IP`/antrean, dan perbandingan jumlah siklus kedua cara penjumlahan.

# Latihan

1. Jelaskan perbedaan arsitektur internal dan arsitektur eksternal sebuah mikroprosesor. Sebutkan dua contoh masing-masing pada 8086.
2. Mengapa 8086 dibagi menjadi BIU dan EU? Jelaskan keuntungannya dibandingkan prosesor dengan satu unit pengolah.
3. Diketahui `CS` = `2A10h` dan `IP` = `00F3h`. Hitung alamat fisik instruksi berikutnya dan tunjukkan langkah perhitungannya.
4. Jelaskan mengapa instruksi `JMP` menghapus keuntungan *instruction queue*. Kaitkan dengan istilah *queue flush*.
5. Di Emu8086, `IP` bertambah tiga setelah `MOV AX, 1234h` tetapi hanya bertambah satu setelah `INC AX`. Benarkah programnya salah? Jelaskan.

# Rangkuman

- Mikroprosesor berkembang dari Intel 4004 (4 bit) hingga 8086 (16 bit) yang menjadi dasar keluarga x86; varian 8088 dipakai pada IBM PC.
- Arsitektur dipandang dari dua sisi: **internal** (register, ALU, unit dalam chip) dan **eksternal** (pin, bus, sinyal kendali).
- 8086 terbagi menjadi BIU dan EU. BIU memegang seluruh lalu lintas bus luar; EU hanya menjalankan instruksi.
- Alamat fisik = (segmen $\times$ 16) + *offset*, sehingga 8086 menjangkau 1 MB meski register-nya 16 bit.
- *Instruction queue* enam byte memungkinkan *pipelining*; lompatan mengosongkan antrean (*queue flush*).
- Tiga kelompok bus: alamat 20-bit (satu arah), data 16-bit (dua arah), kontrol. Siklus instruksi: *fetch–decode–execute–store*.

# Cek Pemahaman

1. Unit yang menghitung alamat fisik 20-bit dari pasangan segmen:offset adalah ...
   - 1.1 *Execution Unit*
   - 1.2 *Bus Interface Unit*
   - 1.3 *Arithmetic Logic Unit*
   - 1.4 register *flag*

2. Ukuran *instruction queue* pada 8086 adalah ...
   - 2.1 2 byte
   - 2.2 4 byte
   - 2.3 6 byte
   - 2.4 16 byte

3. Bila `CS` = `1000h` dan `IP` = `0025h`, instruksi berikutnya diambil dari alamat fisik ...
   - 3.1 `10025h`
   - 3.2 `10250h`
   - 3.3 `1002h`
   - 3.4 `100025h`

## Sebelum pertemuan berikutnya

- Selesaikan praktikum: diagram blok BIU/EU dari ingatan, tabel penelusuran, dan perbandingan jumlah siklus.
- Pelajari kembali hubungan panjang instruksi dengan kenaikan `IP`.
- Baca **Bab V** — Register 8086 dan Register Flag.

**Terima kasih**

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil